

# Documentazione Tecnica

Progetto Realizzato da:

Alberghini Diego, Vona Lorenzo, Iacono Federico, Gariboldi Alessandro, Ghani  
Zeshan





# Software Design Document

---

## Indice

|                                   |        |
|-----------------------------------|--------|
| Introduzione .....                | pag3   |
| Pianificazione Del Progetto ..... | pag4   |
| Organizzazione del team .....     | pag5   |
| Architettura e funzionalità ..... | pag7   |
| Tecnologie utilizzate .....       | pag8   |
| Backend .....                     | pag13  |
| Metodi REST .....                 | pag 21 |



# Software Design Document

---

## Introduzione

Il presente documento descrive la progettazione della webapp *Frutta In Campo*, applicazione web ideata, progettata, creata e testata dal team di sviluppo composto da Alberghini Diego, Vona Lorenzo, Iacono Federico, Gariboldi Alessandro e Ghani Zeshan. Nel Software Design Document è riportato tutto l'andamento del progetto partendo dall'analisi delle richieste del cliente fino ad arrivare alla riuscita creazione dell'applicazione, passando dalla progettazione ed ideazione dei vari compiti e suddividendo le attività da fare in task differenti assegnate in base alle competenze individuali.

La gestione operativa delle attività è stata supportata dall'adozione della metodologia Kanban, implementata tramite la piattaforma Trello, al fine di garantire un monitoraggio continuo dello stato di avanzamento e il rispetto delle scadenze prefissate.

Lo scopo di ***Frutta In Campo*** è creare una webapp comunicante con un server java per la gestione e vendita di prodotti agricoli.

Per quanto riguarda l'amministrazione della produzione si vuole monitorare il ciclo di vita dei prodotti, partendo dalla semina fino ad arrivare al raccolto come richiesto dal Cliente.

In maniera coordinata alla produzione ci sarà la vendita di tali prodotti che saranno esposti nella webapp in vetrina e l'user potrà eseguire l'acquisto sia di prodotti già disponibili, che prenotare (nel caso non ci sia attualmente disponibilità), una data quantità per il futuro.

Il proprietario potrà in base alle circostanze gestire le vendite e gestire la produzione dei prodotti, aggiungendo un nuovo prodotto o un nuovo raccolto per un prodotto già esistente.



# Software Design Document

---

## Pianificazione Del Progetto

Il progetto è stato pianificato in modo da poter garantire a tutti un teamwork ottimale, gestendo e sfruttando al massimo del potenziale tutto il gruppo lavorativo. Come primo passo è stato quello effettivo di creare il gruppo di lavoro; Team formato da:

- 
1. Lorenzo Vona
  2. Diego Alberghini
  3. Federico Iacono
  4. Alessandro Gariboldi
  5. Zeshan Ghani
- 

Prima di aver stabilito chi e quale task doveva eseguire abbiamo creato le task stesse suddivise in compiti in base alle abilità di ognuno di noi.

Abbiamo diviso l'intero progetto nelle seguenti Task:

[Creazione Database, Ideazione Modello e/r, popolamento db, Creazione server java, Costruzione servizio REST/API, Creazione interfaccia JAVA→ DB, Creazione Pagina WEB, implementazione di JS & tecnologia AJAX, Implementazione e realizzazione di front end dinamico, creazione di IMG tagliate da utilizzare nella webapp, Documentazione.](#)

Ogni colore rappresenta un'etichetta e ogni etichetta rappresenta una categoria diversa di lavoro; Di seguito la specifica tecnica di ogni etichetta al suo specifico reparto di lavoro: Reparto di gestione dati in database rappresentato con il colore rosso, reparto script java su rest/api, calcolo su server rappresentato con il colore viola, reparto front end sulla webapp gestita e lavorata su xampp, compreso calcolo di javascript con html,css rappresentato con il colore giallo ed infine il reparto relativo alla documentazione rappresentato dal colore blu.

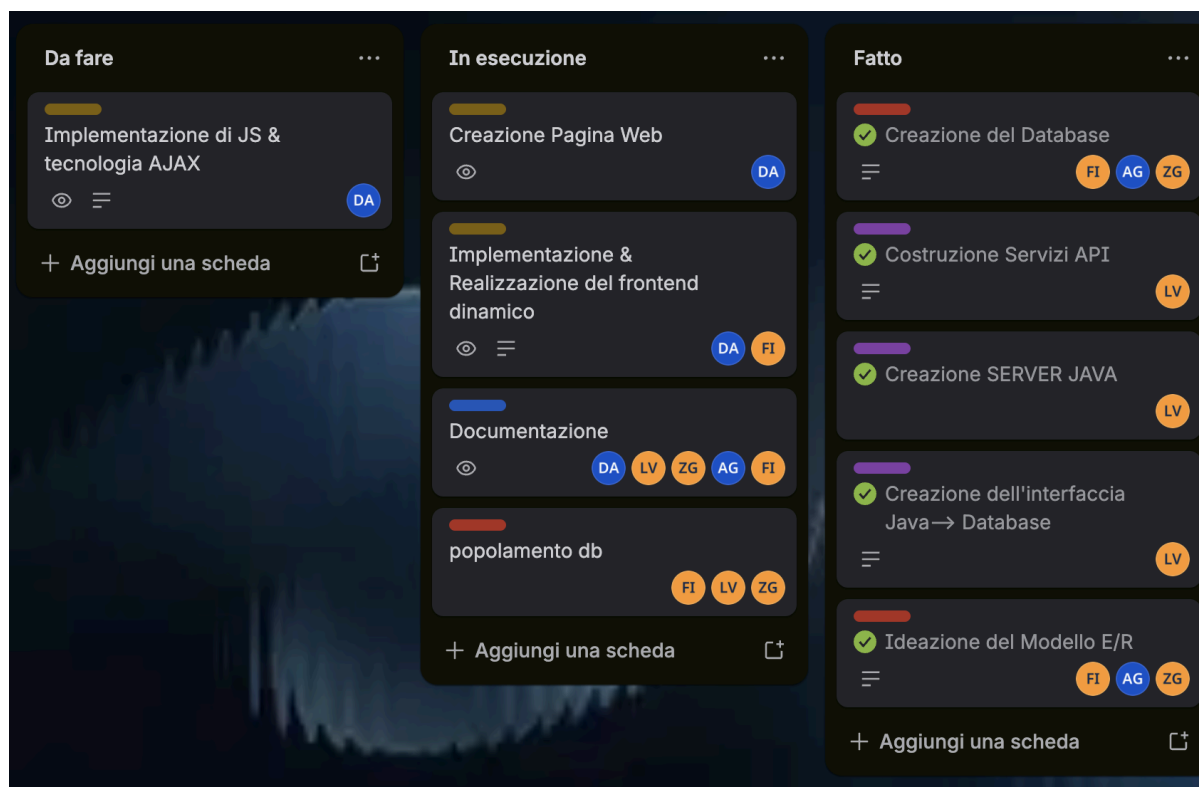
## Organizzazione del team

Passando alla fase dell'organizzazione, il team ha provveduto all'utilizzo di un software online per semplificarsi la gestione/suddivisione e monitoraggio delle varie fasi e task. Il software in questione è **Trello** che consente di sfruttare la metodologia di lavoro Kanban.

Abbiamo creato tre lavagne virtuali: Da fare, In esecuzione, Completate che rappresentano lo stato delle task in tempo reale. L'accesso alla modifica e visualizzazione delle lavagne virtuali è riservato solo ed esclusivamente al personale riservato.

Ogni membro della squadra può cambiare lo stato di un'attività, espressa come “scheda” all'interno di una lavagna, semplicemente spostando questa scheda da una lavagna ad un'altra; Per capire chi e quale attività si sta svolgendo è possibile (e imposto come obbligatorio da noi) assumere e esplicitare nella scheda il membro a cui spetta il lavoro.

Di seguito lasciamo screenshot del processo lavorativo:



esponiamo l'eventuale link d'accesso alla bacheca qui: [bacheca trello](#)



# Software Design Document

All'interno del progetto *Frutta In Campo* il team sviluppativo ha provveduto all'adozione di **GitHub** come sistema di versionamento e collaborazione del codice sorgente.

È stato creato un **repository** centrale rinominato: "aziendaAgricola".

Ogni membro del team per semplificare i comandi di *commit*, *push* e *pull* sarà scaricata l'app desktop: "GitHub Desktop" per usufruire dell'uso del repository tramite interfaccia grafica.

The screenshot shows the GitHub repository page for 'aziendaAgricola' by user 'DiegoAlbe'. The repository is public and has 2 branches and 0 tags. The file list includes 'aziendaAgricolaServer' (4.1.3, yesterday), 'documentazione' (yesterday), 'webApp' (4.1.4, 18 hours ago), '.DS\_Store' (v3.7, 2 days ago), 'README.md' (0.0.0.1, last month), and 'azienda\_agricola.sql' (last week). The README section is titled 'Applicazione Web per Azienda Agricola' and describes the project as a web application for managing and promoting an agricultural business. It lists four main goals: logical correctness without bugs, usability for all users, excellent design, and security. The README also mentions that the application will be developed using web technologies (frontend and backend) with a focus on usability, design, and performance. The right sidebar shows the repository's 'About' section, 'Releases' (no releases published), 'Packages' (no packages published), 'Contributors' (3 contributors: LolloVona, DiegoAlbe, and alegari), and 'Languages' (Java 36.7%, HTML 31.5%, JavaScript 16.1%, CSS 15.5%, PHP 0.2%).



# Software Design Document

---

## Architettura

L'applicazione **Frutta In Campo™** è stata progettata seguendo un'architettura 3-Tier usufruendo di tre livelli quali: *Presentation Layer*, *Application/Business Layer*, *Data Layer*. Questa scelta progettuale ha consentito la facile suddivisione dei compiti del software e la sua visualizzazione in maniera chiara, rendendo il progetto più ordinato e garantendo una facile manutenzione per eventuali modifiche future, al costo di una maggiore complessità di realizzazione e sviluppo.

1. Il primo livello è rappresentato dall'interfaccia utente della web app, realizzata tramite pagine **HTML**, fogli di stile **CSS** e codice **JavaScript**.  
Questo livello ha il compito di mostrare le pagine agli utenti, permettere la navigazione all'interno del sito e raccogliere i dati inseriti nei form, come login, registrazione, gestione prodotti e ordini.
2. Il secondo livello è costituito dal server applicativo sviluppato in **Java con Spring Boot**.  
Questo livello contiene la logica principale dell'applicazione e gestisce le richieste provenienti dal front end tramite **REST API**.  
Quando l'utente compie un'azione, ad esempio effettua l'accesso, visualizza i prodotti o crea un ordine, il front end invia una richiesta al backend, che la elabora, controlla i dati e restituisce una risposta.
3. Il terzo livello è rappresentato dal **database**, utilizzato per memorizzare in modo permanente le informazioni della webapp.  
Al suo interno vengono salvati dati come utenti, prodotti, raccolti, acquisti e ordini.  
Il backend comunica con il database per leggere, inserire, modificare o eliminare i dati necessari al funzionamento del sistema.

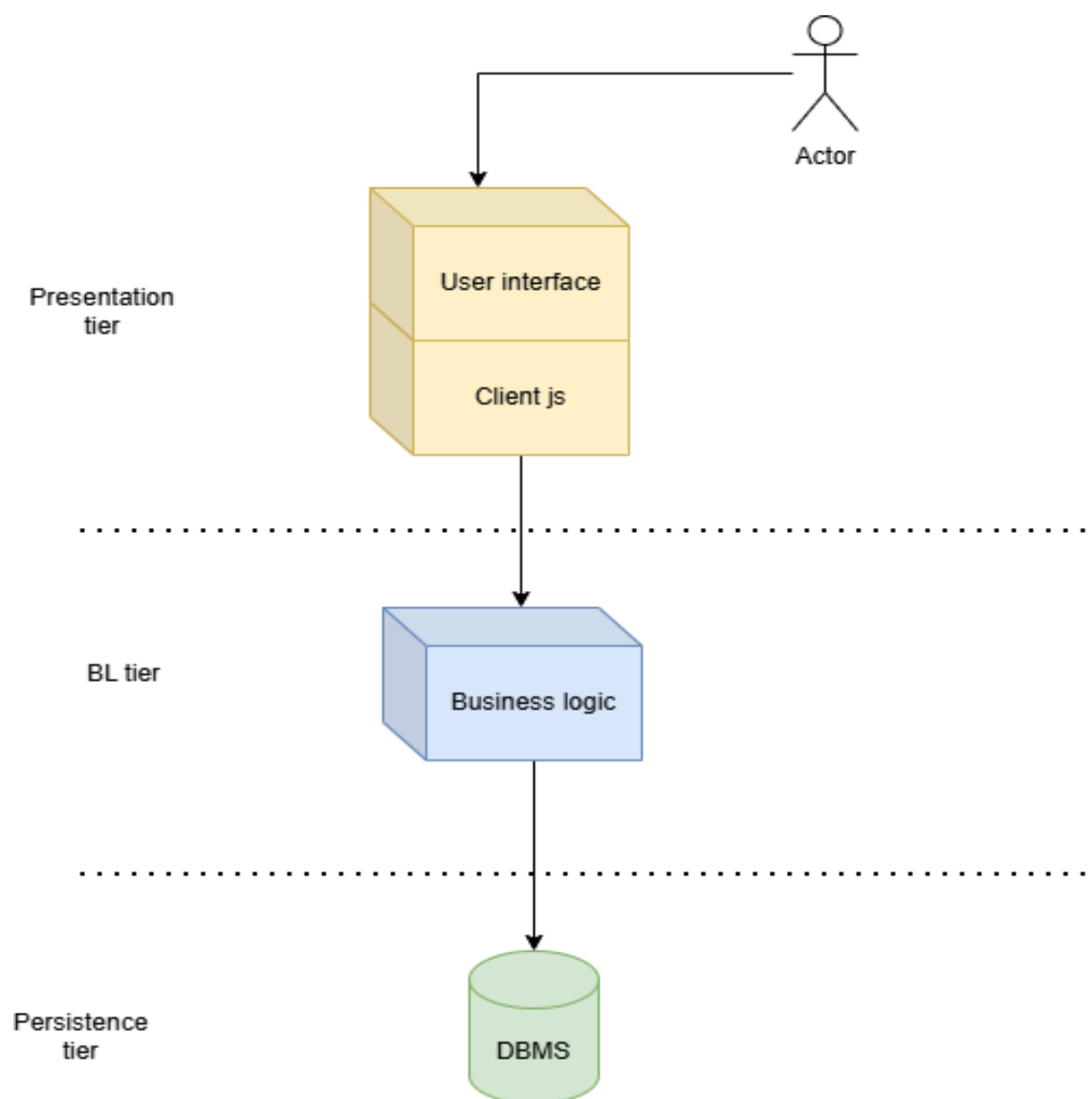
Questo tipo di architettura è risultata come la miglior opzione da intraprendere garantendo numerosi vantaggi per il progetto.

Di seguito nella prossima pagina rilasciamo lo schema grafico dell'architettura 3-tier usufruita con specifici layer utilizzati.

# Software Design Document

---

Inoltre, il progetto vuole consentire la visualizzazione della web app non solo su dispositivi desktop, ma anche su dispositivi mobile perciò è stata implementata nei fogli di stile anche un approccio verso i dispositivi mobili.







# Software Design Document

---

## Tecnologie utilizzate

Per la realizzazione del software *Frutta In Campo*<sup>TM</sup> sono state messe in campo numerose tecnologie, utili al fine corretto del progetto.

### Front End:

Per quanto riguarda la realizzazione del front end sono state utilizzate le seguenti tecnologie:

- **HTML5** → struttura delle pagine web
- **CSS3** → stile grafico, layout responsive e animazioni
- **JavaScript** → gestione dinamica delle pagine e interazione con le API

### Backend

Per la logica applicativa e la gestione delle richieste è stato utilizzato:

- **Java** → linguaggio principale del server
- **Spring Boot** → framework utilizzato per creare le REST API e gestire il backend

### Database

Per la gestione e la memorizzazione dei dati è stato utilizzato:

- **MySQL** → database relazionale del sistema

### Comunicazione Client-Server

La comunicazione tra frontend e backend avviene tramite:

- **REST API**
- **JSON** come formato per lo scambio dei dati

# Software Design Document

---

## Strumenti di sviluppo

Durante lo sviluppo del progetto sono stati utilizzati anche:

- **GitHub** → gestione del versionamento e collaborazione del team
- **Visual Studio Code** → editor di sviluppo del frontend
- **IntelliJ IDEA / Spring Tools** → sviluppo backend Java Spring Boot
- **XAMPP** → ambiente locale di sviluppo e testing
- **Photopea** → webapp online per gestire le img
- **Postman** → SW utilizzato per il testing del backend

# Software Design Document

---

## Database

L'implementazione del database è avvenuta dopo l'idealizzazione di uno schema ER che ha funzionato come base per la sua creazione. Lo schema ER ricavato dall'analisi del problema è risultato come segue:

Da questo grafo abbiamo ricavato i dati necessari alla creazione del database 'azienda agricola' sulla piattaforma phpMyAdmin. Il database presenta 6 tabelle implementate tramite comandi SQL. Le tabelle sono le seguenti con i propri attributi.

La tabella **acquisto** registra le operazioni di vendita o acquisto effettuate, permette di archiviare e gestire le transazioni economiche dell'azienda agricola.

Attributi:

- **Numero\_fattura**: Numero univoco del documento fiscale (Chiave Primaria).
- **Data\_acquisto**: Giorno in cui è avvenuto l'acquisto.
- **Totale**: Importo complessivo della fattura.
- **Data\_erogazione**: giorno in cui è pronto l'ordine

La tabella **aggiornamento** registra tutte le modifiche effettuate nel sistema, garantisce la tracciabilità delle operazioni eseguite dagli utenti, implementando un sistema di storico e controllo delle modifiche.

Attributi:

- **Id\_aggiornamento**: Identificativo della singola operazione di modifica (Chiave Primaria).
- **Tipo\_aggiornamento**: Natura della modifica effettuata.
- **Data\_aggiornamento**: Momento in cui è stato eseguito l'aggiornamento.
- **Attributo\_modificato**: Nome del campo che è stato variato.
- **Vecchio\_valore**: Il valore dell'attributo prima della modifica.
- **Nuovo\_valore**: Il valore dell'attributo dopo la modifica.

La tabella **prodotto** memorizza le informazioni sui prodotti agricoli gestiti dall'azienda, consente di monitorare le scorte presenti e i dati commerciali dei prodotti.

Attributi:

- **Id\_prodotto**: Codice identificativo del prodotto (Chiave Primaria).
- **Nome\_prodotto**: Nome del prodotto.
- **Prezzo**: Valore monetario unitario.
- **Magazzino**: Quantità attualmente stoccata.
- **Disponibilita**: Quantita di prodotto disponibile.

La tabella **prossimoraccolto** contiene le informazioni relative ai raccolti programmati, permette la pianificazione della produzione agricola e il monitoraggio dei futuri raccolti.

Attributi:

- **Id\_raccolto**: Identificativo univoco del lotto previsto (Chiave Primaria).

# Software Design Document

---

- **Data\_raccolto**: Data prevista per il raccolto.
- **Totale**: Quantità complessiva stimata del raccolto.
- **Disponibilit \_raccolto**: Parte del raccolto effettivamente prenotabile o disponibile.

La tabella **relativo** rappresenta la relazione tra acquisti e prodotti, implementa una relazione multi-a-molti tra acquisti e prodotti, consentendo di associare pi  prodotti a una singola fattura e viceversa.

Attributi:

- **Numero\_fattura**: Numero univoco del documento fiscale (Chiave Esterna).
- **Id\_prodotto**: Identificativo univoco del prodotto (Chiave Esterna).
- **Quantita**: Quantit  di prodotti relativi alla fattura.

La tabella **utente** contiene le informazioni relative agli utilizzatori del sistema gestionale, permette la gestione degli accessi e dei permessi all'interno della piattaforma.

Attributi:

- **Id\_utente**: Identificativo univoco dell'utente (Chiave Primaria).
- **Nome**: Nome completo,
- **Username**: Nome utente utilizzato per l'accesso.
- **Password**: Chiave d'accesso associata all'account.
- **Tipo**: Ruolo dell'utente (Amministratore o Cliente).

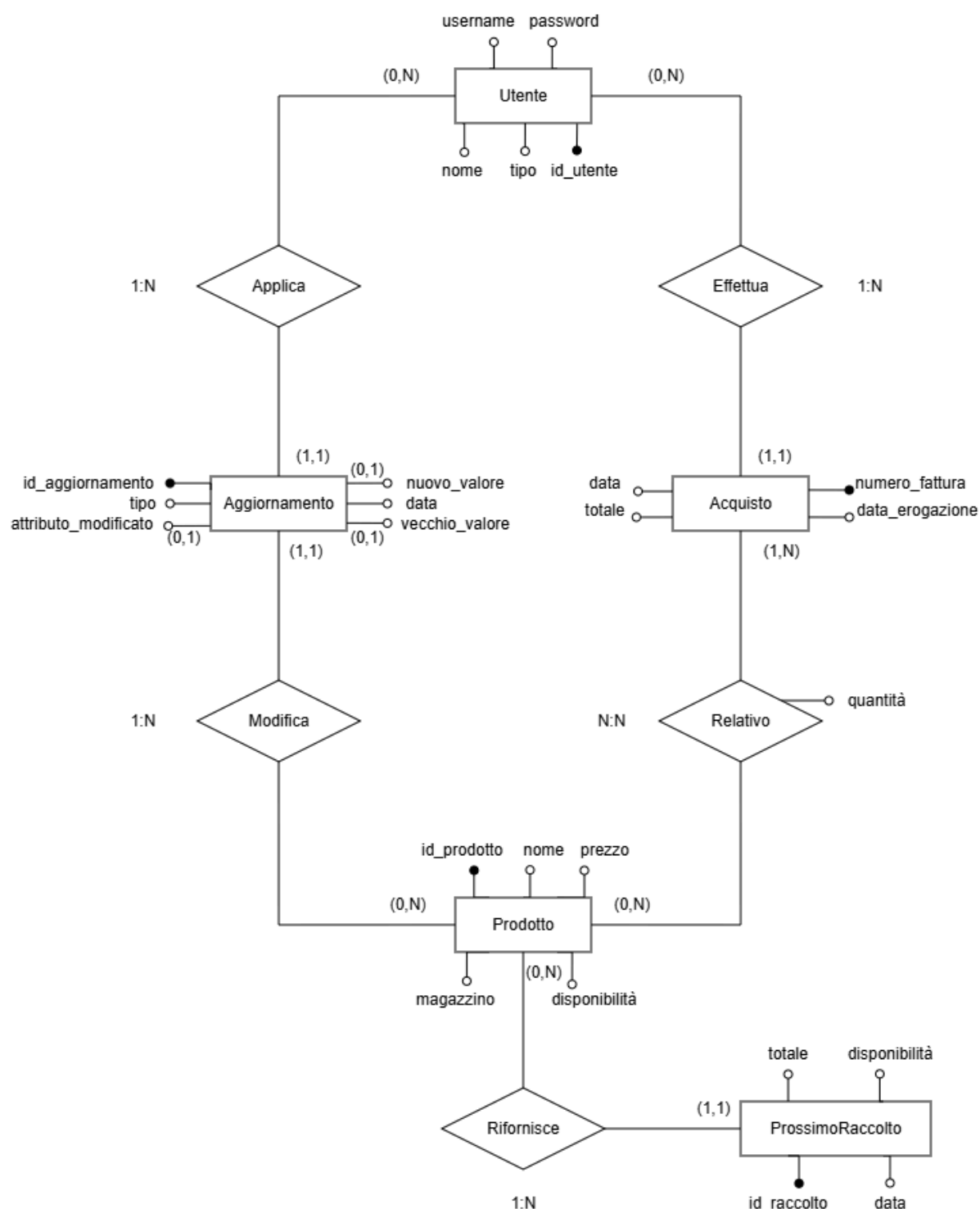
Il database utilizza relazioni relazionali per mantenere la coerenza dei dati:

- Un **utente** pu  effettuare pi  aggiornamenti.
- Un **prodotto** pu  comparire in pi  raccolti futuri.
- Un **prodotto** pu  essere associato a pi  fatture.
- Una **fattura** pu  contenere pi  prodotti.
- Ogni modifica registrata nella tabella aggiornamento   associata sia a un utente sia a un prodotto.

L'implementazione del database consente all'azienda agricola di:

- gestire in modo efficiente prodotti e magazzino;
- monitorare vendite e fatture;
- pianificare i raccolti futuri;
- controllare le modifiche effettuate dagli operatori;
- migliorare l'organizzazione e la tracciabilit  delle attivit  aziendali.

# Software Design Document





# Software Design Document

---

Vincoli:

V1: Utente.tipo={'C','A'}

V2: Aggiornamento.tipo={'C','U','D'}

V3: Prodotto.prezzo>0

V4: Prodotto.magazzino>=0

V5: Prodotto.disponibilità>=0

V6: Prodotto.magazzino>=Prodotto.disponibilità

V7: ProssimoRaccolto.totale>0

V8: ProssimoRaccolto.disponibilità>=0

V9: Relativo.quantità>=0

V10: Acquisto.totale>0

A seguito della ristrutturazione si sono stabilite seguenti relazioni:

UTENTE(Id\_utente(PK), Nome, Username, Password, Tipo)

AGGIORNAMENTO(Id\_aggiornamento(PK), Tipo, Data, Attributo\_modificato\*,  
Nuovo\_valore\*, Vecchio\_valore\*, Id\_utente(FK), Id\_prodotto(FK))

PRODOTTO(Id\_prodotto(PK), Nome, Prezzo, Magazzino, Disponibilità)

ACQUISTO(Numero\_fattura(PK), data, totale, data\_erogazione, id\_utente(FK))

RELATIVO((Numero\_fattura(FK), Id\_prodotto(FK))(PK), Quantità)

PROSSIMORACCOLTO(Id\_raccolto(PK), Disponibilità, Totale, Data, Id\_prodotto(FK))

# Software Design Document

---

## Backend

Il sistema software sviluppato per la gestione dell'azienda agricola si basa sul pattern architetturale **Layered Architecture**.

L'obiettivo primario di questa organizzazione in pacchetti (package) è il perseguimento di un elevato **disaccoppiamento** tra le componenti e il rispetto del **Principio di Singola Responsabilità (Single Responsibility Principle)**. Ciascun pacchetto isola una specifica funzionalità del ciclo di vita del dato, rendendo l'applicazione scalabile, manutenibile e facilmente sottoponibile a test unitari.

### Package entità (Modello del Dominio e Persistenza)

Il pacchetto entità costituisce il cuore informativo del sistema. Contiene le classi Java (come Prodotto, Acquisto, Relativo e ProssimoRaccolto) che mappano direttamente le tabelle del database relazionale attraverso lo standard **JPA (Java Persistence API)** e Hibernate. Definisce la struttura dei dati permanenti del sistema. Le classi sono decorate con annotazioni dichiarative (@Entity, @Table, @Id, @Column) e gestiscono i vincoli di integrità referenziale tramite relazioni @ManyToOne e @OneToMany. Questo strato non contiene logica algoritmica, ma si limita alla rappresentazione dello stato del dominio tramite costruttori e metodi di accesso (Getter/Setter).

### Package repository (Data Access Layer)

Questo strato funge da mediatore tra il modello a oggetti in Java e il database relazionale sottostante. Contiene interfacce dedicate all'esecuzione delle operazioni di persistenza.

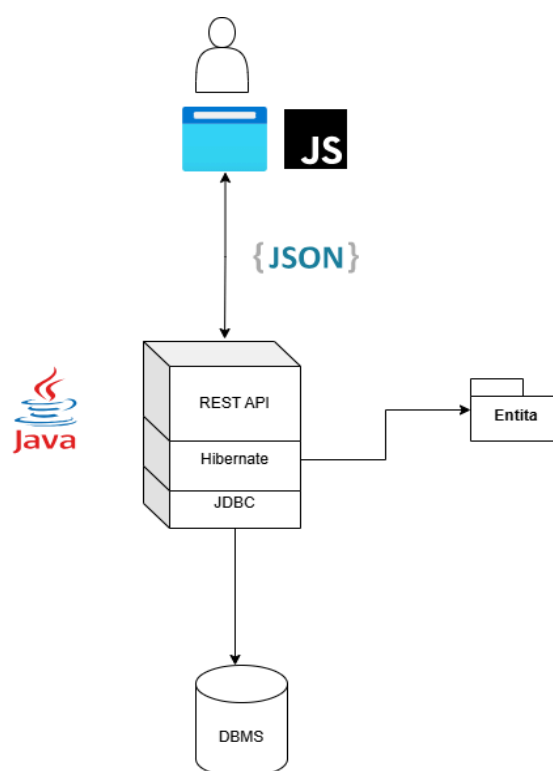
La sua funzione è quella di isolare i livelli superiori dalle specificità del database e dai dettagli implementativi del linguaggio SQL.

Sfruttando le potenzialità di Spring Data JPA, questo livello consente di effettuare query e mutazioni sul database in modo astratto, garantendo che un eventuale cambio di DBMS (Database Management System) non impatti la logica di business dell'applicazione.

L'estensione di JpaRepository consente all'interfaccia di ereditare immediatamente una suite completa di operazioni **CRUD** standard (save(), findById(), findAll(), deleteById()).

Il framework analizza la firma e la nomenclatura dei metodi dichiarati nell'interfaccia derivata, traducendoli autonomamente in query SQL a runtime.

### Package service (Business Logic Layer)



# Software Design Document

---

Il pacchetto service rappresenta il motore computazionale dell'applicazione, all'interno del quale risiedono le regole di business e gli algoritmi di controllo dell'azienda agricola. La sua funzione è quella di elaborare i dati, coordinare i flussi di lavoro ed eseguire calcoli complessi.

In questo strato sono implementate le funzioni critiche di allineamento delle scorte, come i processi di incremento e decremento delle giacenze di magazzino in base ai raccolti o agli acquisti evasi. I componenti di questo livello (es. AcquistoService) integrano logiche di validazione preventiva e gestiscono l'atomicità delle operazioni mediante la gestione delle transazioni (@Transactional).

## **Package dto (Data Transfer Object)**

Il pacchetto dto contiene oggetti di puro trasporto dati, strutturati per ottimizzare lo scambio di informazioni tra il client esterno (interfaccia utente) e il backend.

Sfoltisce le entità da dati sovrabbondanti e implementa criteri di sicurezza informatica.

L'adozione dei DTO (es. AcquistoDTO) impedisce l'esposizione diretta delle entità del database verso l'esterno. Ciò scherma l'applicazione da vulnerabilità di sicurezza e consente l'applicazione di regole di validazione formale in input (es. @NotNull, @Min) prima che i dati raggiungano lo strato di business.

## **Package controller (Web / REST API Layer)**

Rappresenta l'interfaccia pubblica del backend, esponendo gli endpoint di comunicazione a cui i client si collegano tramite protocollo HTTP.

Riceve le richieste di rete, le indirizza allo strato logico e formatta le risposte.

I componenti (es. AcquistoController), annotati con @RestController, intercettano i metodi HTTP (GET, POST, ecc.), convertono i payload JSON in oggetti di tipo DTO e delegano l'elaborazione al Service competente, restituendo infine al client la risposta serializzata accompagnata dal corretto codice di stato HTTP.

## **Package record e log**

Il package contenente record è stato utilizzato per inserire tutti i record poi necessari per le response.

Il package dei log invece contiene le classi per la scrittura dei file di log

Sono state implementate le seguenti funzionalità (si rimanda alla sezione metodi rest per tutte le informazioni su come invocare i servizi):

1. Registrazione utente: aggiunge un nuovo utente. Restituisce l'idUtente nel caso vada a buon fine
2. Accesso: effettua l'accesso. Restituisce l'idUtente nel caso vada a buon fine
3. Aggiunta prodotto: aggiunge un nuovo prodotto. La disponibilità iniziale viene settata allo stesso valore del magazzino, tipo è "C".



# Software Design Document

---

4. Eliminazione prodotto: dato il nome del prodotto (così come è stato fornito) lo elimina. Il nome è case sensitive. Il nome del prodotto va specificato nell'URI.
5. Get di tutti i prodotti: restituisce le informazioni di tutti i prodotti salvati nel database
6. Get di un prodotto da nome: restituisce le informazioni del prodotto il cui nome è stato inserito nell'uri. Il nome è case sensitive
7. Modifica nome prodotto: dato il nome di un prodotto (nel body) e il nuovo nome (nell'URI) modifica il nome del prodotto. Il metodo è case sensitive
8. Modifica prezzo prodotto: dato il nome di un prodotto (nel body) e il nuovo prezzo(nell'URI) modifica il prezzo del prodotto. Il metodo è case sensitive
9. Aggiunta raccolto: dato il nome di un prodotto, l'id dell'utente, il totale del raccolto e la data di raccolta, aggiunge un nuovo raccolto
10. Ordine di uno o più prodotti: dato id utente e un vettore di oggetti contenenti il nome del prodotto e la quantità restituisce le informazioni, per confermare l'ordine occorrerà mandare una richiesta al metodo di conferma dell'ordine
11. Conferma dell'ordine di una o più informazioni: conferma l'ordine salvando le informazioni nel database
12. Visualizza gli ordini da erogare: restituisce le informazioni su tutti gli ordini ancora da erogare
13. Tipo utente da ID: dato l'id dell'utente restituisce il tipo (C o A)

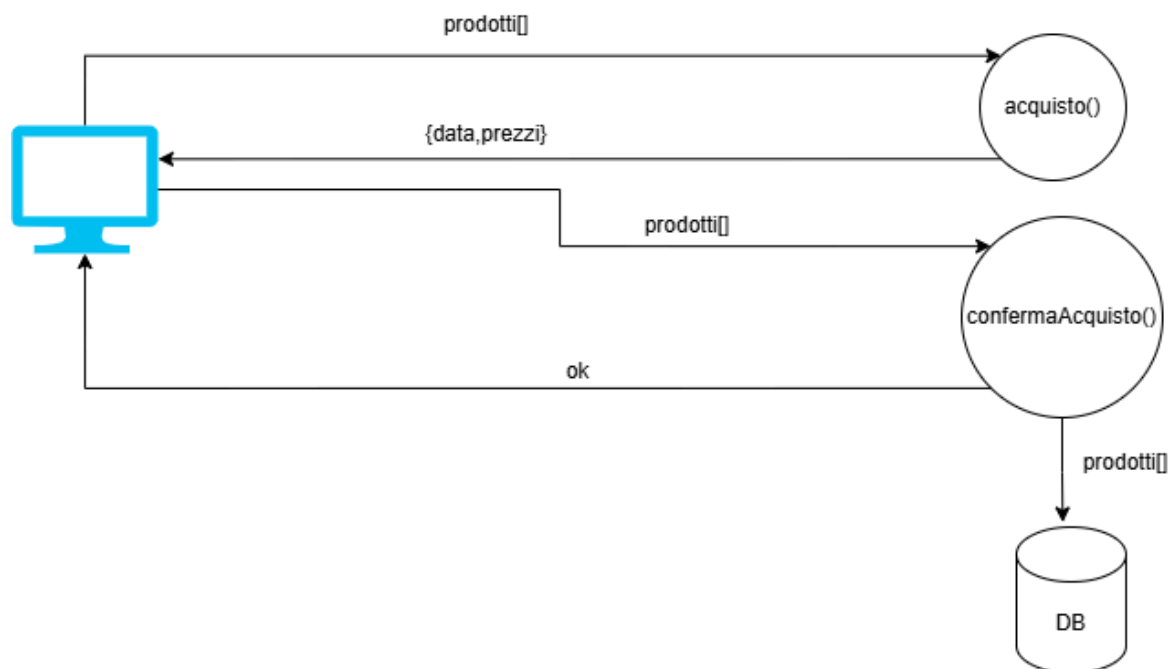
È presente, inoltre, una task schedulata si attiva quotidianamente alle ore 00:00 e al boot del server per allineare l'inventario dei prodotti alla data corrente. L'algoritmo elabora gli acquisti registrati riducendo le quantità in magazzino e, contestualmente, carica i nuovi raccolti giunti a scadenza incrementando la disponibilità del prodotto e aggiornando il magazzino.

Data la complessità logica che caratterizza l'acquisto dei prodotti, il processo viene approfondito di seguito. L'operazione si suddivide in due step sequenziali:

- Fase di simulazione/ordine: Viene invocato il metodo dedicato che verifica la disponibilità del prodotto. Se la quantità richiesta è soddisfatta, il sistema restituisce il prezzo e la data di erogazione. Tale data viene calcolata individuando il momento in cui la disponibilità attuale del prodotto, sommata alle disponibilità dei futuri raccolti associati, eguaglia o supera il quantitativo richiesto.
- Fase di conferma: Se il prezzo e la data di erogazione soddisfano i requisiti del cliente, quest'ultimo può procedere con la conferma dell'ordine, che formalizza l'acquisto effettivo.

# Software Design Document

---



Per visualizzare il diagramma delle classi andare nella cartella “*Diagrammi*” e aprirlo nel formato preferito.

Il Diagramma delle classi è stato semplificato in quanto rappresentare tutte le associazioni avrebbe significato un inutile complicazione del diagramma.

È suggerita l’aggiunta delle seguenti funzionalità in una eventuale futura nuova versione:

- funzionalità pagamento online
- notifica quando ordine è pronto
- notifica quando è pronto raccolto
- sicurezza:
  - Password crittografata
  - Modifiche al DB fatte in base a token di identificazione



# Software Design Document

---

## Frontend

Per quanto riguarda la sviluppazione del reparto di **front end** sono state suddivise le aree di lavoro in differenti cartelle. La cartella madre del progetto è stata rinominata **webapp** con al suo interno tutti file necessari al corretto funzionamento di tutti i servizi front end.

All'interno della cartella backend saranno presenti il file "padre" di tutti gli html (homepage) "index.html" e sottocartelle di lavoro "Assets" e "Campo".

All'interno della sottocartella campo sono presenti solo ed esclusivamente gli altri file html quali:

- admin.html
- catalogo.html
- client.html
- login.html
- paid.html
- chiSiamo.html

Ed infine l'ultima sottocartella "reserved" che contiene i file html riguardanti le pagine destinate all'organizzazione dell'admin:

- ordini.html
- prodotto.html
- raccolto.html

Per quanto riguarda la cartella "Assets" al suo interno è stata ulteriormente suddivisa nei vari assets utilizzati dalla webapp, quali:

- Css
- Img
- Js
- Php

1. All'interno di **css** sono presenti 6 file differenti css, per non utilizzare un solo file pesante di stile, e ogni stylesheet fa riferimento ad una pagina html ben precisa e commentata nel codice.

File presenti:

- admin.css
- client.css
- log.css

# Software Design Document

---

- ordini.css
- style.css
- thanks.css

2. All'interno della sottocartella img sono presenti le immagini per forza da scaricare all'interno dello storage quali:

- 1.png
- logo.svg
- icon.ico
- team.png
- IMG DEI PRODOTTI [elenco non disponibile in quanto aggiunte dinamicamente ]

3. All'interno della sottocartella js sono presenti tutti i file di linguaggio di tipo javascript utilizzando la tecnologia Ajax comunicanti con web service tramite interfaccia REST/API

- admin.js
- catalogo.js
- client.js
- inside.js
- ordini.js
- raccolto.js
- script.js

4. All'interno della sottocartella php è presente un unico file php utile allo spostamento dell'img del prodotto (in maniera dinamica) quando si aggiunge l'img di un nuovo prodotto

- upload.php

Ogni file è precisamente commentato per garantire una corretta comprensibilità delle istruzioni eseguite al proprio interno.

È importante sottolineare come il team di sviluppo front end abbia provveduto alla comunicazione dei web service tramite REST/API. Il metodo javascript utilizzato è il metodo `fetch()` all'interno di funzioni asincrone richiamate dalla webapp stessa. I metodi comunicanti con tecnologia Ajax, dunque con REST/API sono **tutti** asincroni per garantire comunque la funzionalità della webapp nonostante ci sia un guasto nella comunicazione; Questo è disponibile grazie anche al comando `"await"` prima di ogni metodo `fetch()`.

# Software Design Document

---

Di sotto riportati esempi di codice generalizzato per i metodi principali (post) e (get)

## Get - chiede dati:

```
async function caricaProdotti() {  
    const response = await fetch("http://localhost:8080/api/prodotto",  
    {  
        method: "GET",  
        headers: {  
            "Content-Type": "application/json"  
        }  
    })  
  
    const message = await response.json();  
    mostraProdotti(message.prodotti);  
}
```

## Get - per modifica dati *(per modifiche)*

```
async function elimina(nome) {  
  
    const id = sessionStorage.getItem("idUtente");  
    const response = await fetch(  
        `http://localhost:8080/api/prodotto/${encodeURIComponent(nome)}`,  
        {  
            method: "DELETE",  
            headers: {  
                "Content-Type": "application/json"  
            },  
            body: JSON.stringify({  
                idUtente: Number(id)  
            })  
        })  
  
    if(response.status == 204) {  
        alert('prodotto eliminato con successo');  
        location.reload();  
    } else {  
        alert(err);  
    }  
}
```

# Software Design Document

---

## Post - invio dati al server (*es: login*)

```
async function accedi() {
    var us = document.getElementById('us').value;
    var pw = document.getElementById('pw').value;

    //devo mandare i dati al server
    const response = await fetch("http://localhost:8080/api/utente/accesso",
        {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify({
                username: us,
                password: pw
            })
        });

    //verifico che il server mi abbia risposto:
    const message = await response.json();

    switch(response.status) {
        case 201:
            sessionStorage.setItem("idUtente", message.idUtente);
            reindirizzamento();
            break;
        case 400:
            document.getElementById('err').style.display = "block";
            document.getElementById('err').innerText = "Error server code: 400";
            break;
    }
}
```



# Software Design Document

---

## Metodi REST

| Azione                                         | Metodo HTTP | URI                                                                                                                   | Metodo Server                                 |
|------------------------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Registrazione utente                           | POST        | <a href="http://localhost:8080/api/utente/registrazione">http://localhost:8080/api/utente/registrazione</a>           | UtenteController.registra()                   |
| Accesso                                        | POST        | <a href="http://localhost:8080/api/utente/accesso">http://localhost:8080/api/utente/accesso</a>                       | UtenteController.accedi()                     |
| Aggiunta prodotto                              | POST        | <a href="http://localhost:8080/api/prodotto">http://localhost:8080/api/prodotto</a>                                   | UtenteController.aggiungi()                   |
| Eliminazione prodotto                          | DELETE      | <a href="http://localhost:8080/api/prodotto/{nomeProdotto}">http://localhost:8080/api/prodotto/{nomeProdotto}</a>     | UtenteController.elimina()                    |
| Get di tutti i prodotti                        | POST        | <a href="http://localhost:8080/api/prodotto">http://localhost:8080/api/prodotto</a>                                   | UtenteController.getProdotti()                |
| Get di un prodotto da nome                     | DELETE      | <a href="http://localhost:8080/api/prodotto/{nomeProdotto}">http://localhost:8080/api/prodotto/{nomeProdotto}</a>     | UtenteController.elimina()                    |
| Modifica nome prodotto                         | PUT         | <a href="http://localhost:8080/api/prodotto/nome/{nuovoNome}">http://localhost:8080/api/prodotto/nome/{nuovoNome}</a> | UtenteController.cambiaNome()                 |
| Modifica prezzo prodotto                       | PUT         | <a href="http://localhost:8080/api/prodotto/prezzo/{prezzo}">http://localhost:8080/api/prodotto/prezzo/{prezzo}</a>   | UtenteController.cambiaPrezzo()               |
| Aggiunta raccolto                              | POST        | <a href="http://localhost:8080/api/raccolto">http://localhost:8080/api/raccolto</a>                                   | ProssimoRaccoltoController.aggiungiRaccolto() |
| Ordine di uno o più prodotti                   | POST        | <a href="http://localhost:8080/api/acquisto">http://localhost:8080/api/acquisto</a>                                   | AcquistoController.acquisto()                 |
| Conferma dell'ordine di una o più informazioni | POST        | <a href="http://localhost:8080/api/acquisto/conferma">http://localhost:8080/api/acquisto/conferma</a>                 | AcquistoController.confermaAcquisto()         |
| Visualizza gli ordini da erogare               | GET         | <a href="http://localhost:8080/api/acquisto">http://localhost:8080/api/acquisto</a>                                   | AcquistoController.getOrdiniDaErogare()       |
| Tipo utente da ID                              | GET         | <a href="http://localhost:8080/api/utente/tipo">http://localhost:8080/api/utente/tipo</a>                             | UtenteController.tipo()                       |



# Software Design Document

---

| Azione                     | Body request                                                   | Body response                                                                                         | Codici possibili                                                                              |
|----------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Registrazione utente       | username:String<br>password:String<br><br>nome:String          | idUtente:int<br>message:String                                                                        | 201: Utente creato<br>400: Dati errati<br>409: Utente già presente                            |
| Accesso                    | username:String<br>password:String                             | idUtente:int<br>message:String                                                                        | 201: Accesso effettuato<br>400: Dati errati<br>401: Credenziali sbagliate                     |
| Aggiunta prodotto          | idUtente:int<br>nome:String<br>prezzo:float<br>magazzino:float | message:String                                                                                        | 201: Prodotto creato<br>400: Dati errati<br>400: IdUtente mancante<br>403: Non autorizzato    |
| Eliminazione prodotto      | idUtente:int                                                   | message:String                                                                                        | 204: Prodotto eliminato<br>400: Dati errati<br>400: IdUtente mancante<br>403: Non autorizzato |
| Get di tutti i prodotti    |                                                                | prodotti: [<br>{<br>{<br>nome: String,<br>prezzo: float,<br>disponibilita: int<br>}<br>},<br>...<br>] | 200: Ok                                                                                       |
| Get di un prodotto da nome |                                                                | prodotto: {<br>nome: String,<br>prezzo: float,<br>disponibilita: int<br>}<br>message:String           | 200: Ok<br>400: Dati non validi                                                               |





# Software Design Document

---

|                                                |                                                                                      |                                      |                                                                                                |
|------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------|------------------------------------------------------------------------------------------------|
| Modifica nome prodotto                         | nome:String<br>idUtente:int                                                          | message:String                       | 204: Prodotto modificato<br>400: Dati errati<br>400: IdUtente mancante<br>403: Non autorizzato |
| Modifica prezzo prodotto                       | nome:String,<br>idUtente:int                                                         | message:String                       | 204: Prodotto modificato<br>400: Dati errati<br>400: IdUtente mancante<br>403: Non autorizzato |
| Aggiunta raccolto                              | totale:int<br>data:date<br>nome:String<br>idUtente:int                               | message:String                       | 204: Raccolto creato<br>400: Dati errati<br>400: IdUtente mancante<br>403: Non autorizzato     |
| Ordine di uno o più prodotti                   | idUtente:int<br>prodotti:[<br>{<br>nome:String,<br>quantita:float<br>},<br>....<br>] | prezzo:float,<br>dataErogazione:Date | 200:ok<br>400:dati errati                                                                      |
| Conferma dell'ordine di una o più informazioni | idUtente:int<br>prodotti:[<br>{<br>nome:String,<br>quantita:float<br>},<br>....<br>] | message:String                       | 200:Acquisto effettuato<br>400:dati errati                                                     |



# Software Design Document

---

|                                  |  |                                                                                                                                     |                               |
|----------------------------------|--|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Visualizza gli ordini da erogare |  | <pre>ordini: [   {     numeroFattura:int,     totale:float,     dataErogazione:date,      usernameCliente:String   },   ... ]</pre> | 200:ok<br>403:non autorizzato |
| Tipo utente da ID                |  | <pre>tipo:String</pre>                                                                                                              | 200:ok<br>400:Errore nei dati |